Project SentinelCare — Founder Blueprint

1. Executive Summary

Project SentinelCare is a multimodal, real-time patient-safety intelligence platform designed to detec predict, prevent, and learn from medical errors across healthcare settings. It is not a single r model. It is a layered safety system composed of: - a healthcare interoperability and event-ing backbone - clinical knowledge and policy engines - multimodal detection and prediction services intervention and escalation workflows - audit, governance, validation, and model-ops controls - hur oversight at every high-risk decision point

The first deployable version should focus on three measurable pilot domains: 1. medication safety critical-result follow-up safety 3. deterioration / sepsis / shock surveillance

2. Vision

Build an always-on safety copilot that continuously monitors care processes, surfaces latent risks, deviations, predicts preventable harm, and recommends safer next actions before injury occurs.

3. Mission

Reduce preventable harm by making patient safety computable, observable, actionable, and continuou improvable.

4. Product Principles

1. Human-in-the-loop by default
2. Evidence-backed alerts only
3. High precision before interruption
4. Fewer, better alerts over maximal alerts
5. Workflow-native, not dashboard-only
6. Measurable impact on safety outcomes
7. Auditability, traceability, and override logging
8. Privacy, security, and regulatory readiness from day one
9. Site adaptability without losing governance
10. Modular architecture for expansion across specialties and settings

5. Problem Statement

Healthcare errors arise from failures of omission, commission, timing, sequencing, communication, foll up, and system design. Current safety systems are fragmented, retrospective, and narrow. Most to not combine structured clinical data, text, device signals, images, audio, operations data, and wo context into one prospective safety layer.

6 . Product Scope

SentinelCare should ultimately cover: - hospitals - ICUs - emergency departments - operating roo pharmacies - laboratories - radiology - ambulatory clinics - home care - ambulances - public-h programs - administrative / operational pathways

7 . Safety Domains

7 . 1 Medication Safety

- allergy conflicts
- dose errors
- renal/hepatic adjustment omissions
- duplication
- interactions
- omitted critical meds
- discharge medication mismatch

7 . 2 Diagnostic Safety

- delayed diagnosis
- unacknowledged abnormal tests
- discordant findings
- follow-up failures
- referral leakage

7 . 3 Procedure / Surgical Safety

- wrong patient / site / procedure
- consent mismatch
- checklist non-compliance
- device readiness gaps
- sterility breaks

7 . 4 Deterioration Surveillance

- sepsis
- shock
- hypoxia
- AKI
- postpartum emergencies
- hemorrhage
- delirium

7 . 5 IPC and Environmental Safety

- isolation failures

- line / catheter risk patterns
- PPE or sterility workflow deviations
- outbreak weak-signal detection

7.6 Lab / Radiology Process Safety

- sample mismatch
- critical value delays
- missed acknowledgments
- report-to-action gaps

7.7 Care Transition Safety

- discharge follow-up gaps
- handover omissions
- referral failures
- medication reconciliation failures

7.8 Documentation / Communication Safety

- contradictions across notes
- ambiguous verbal orders
- undocumented red flags
- responsibility ambiguity

7.9 Device / Equipment Safety

- infusion pump mismatch
- ventilator alarm pattern risk
- maintenance and calibration issues
- sensor disconnections

7.10 Operational Safety

- staffing mismatch
- transport delays
- overload conditions
- blood product logistics gaps
- queue-related care delays

8. Core User Roles

- bedside nurse
- duty doctor / resident
- consultant
- pharmacist
- lab physician / technician
- radiologist

- OT nurse / surgeon / anesthetist
- infection control nurse
- safety officer / quality team
- hospital administrator
- biomedical engineer
- public-health officer
- patient / caregiver (select workflows)

9. Foundational Design Model

Represent each clinical process as: **Intent -> Action -> Confirmation -> Response -> Follow-up -> Closure**

An error is any unsafe deviation, missing expected action, contradictory evidence, or harmful delay this chain.

10. Platform Architecture

Layer 1: Data Fabric

- HL7/FHIR/DICOM connectors
- ETL and stream ingestion
- event normalization
- identity resolution
- time synchronization
- terminology mapping

Layer 2: Clinical Knowledge Layer

- guidelines and SOP library
- formulary rules
- contraindication logic
- escalation policies
- checklist logic
- site-specific governance versioning

Layer 3: Detection Layer

- deterministic rules engine
- anomaly detection engine
- process deviation engine
- contradiction engine
- NLP extraction engine
- vision / audio extractors

Layer 4: Prediction Layer

- deterioration models

- ADE risk models
- missed follow-up models
- diagnostic delay models
- operational risk forecasts

Layer 5 : Intervention Layer

- silent signals
- nudges
- interruptive alerts
- escalation workflows
- checklist launchers
- one-click corrective actions

Layer 6 : Learning Layer

- override learning
- feedback collection
- false-positive and false-negative review
- site drift detection
- post-event learning loops

Layer 7 : Governance Layer

- model registry
- audit logs
- approval workflows
- alert policy management
- explainability store
- rollback / kill switch

Layer 8 : Experience Layer

- clinician worklist UI
- role-specific dashboards
- mobile alerts
- patient-safety command center
- admin console

11. Data Inputs

Passive Inputs

- EMR / EHR
- orders and MAR
- lab results
- radiology reports

- nursing notes
- monitor streams
- device data
- discharge summaries
- staffing rosters
- checklists
- audit logs
- inventory and maintenance records
- claims / billing anomalies

Active Inputs

- confirmations
- structured safety prompts
- barcode scans
- bedside photo / video
- voice handovers
- patient symptom reports
- smart checklists
- wearable streams

12. Product Modules

1. Interoperability hub
2. Universal event store
3. Safety ontology service
4. Clinical rules engine
5. Multimodal extraction service
6. Real-time scoring engine
7. Alert orchestration service
8. Escalation / notification service
9. Incident learning service
10. Feedback and override capture
11. Governance console
12. Evaluation and validation workbench
13. Model registry and deployment manager
14. Analytics / KPI dashboard
15. Human review queue
16. Simulation / synthetic testing lab

13. MVP Definition

MVP-A: Medication Safety Engine

Required outputs: - hard contraindication alerts - dose-adjustment warnings - duplicate therapy fla omitted-high-risk-med suggestions - discharge med reconciliation checks

MVP-B: Critical Result Closure Engine

Required outputs: - critical result unacknowledged alerts - pending result after discharge flags - documented-action reminders - escalation to responsible team

MVP-C: Deterioration Surveillance Engine

Required outputs: - rising-risk watchlist - sepsis bundle deviation alerts - delayed escalation flags doctor role-based action prompts

14. PRD

Product Goals

- reduce preventable medication incidents
- reduce missed critical result follow-up
- improve early recognition and escalation of deterioration
- reduce alert fatigue using precision-oriented design
- provide auditable and explainable outputs

Non-goals (v1)

- autonomous diagnosis confirmation
- autonomous treatment ordering
- unrestricted autonomous patient management
- replacing clinician judgment

Key Jobs To Be Done

- tell the right person the right risk at the right time
- catch missing actions and risky deviations before harm occurs
- make the system explain why it is alerting
- support quick correction inside workflow
- learn from clinician feedback

Success Metrics

- PPV of alerts
- sensitivity for selected safety events
- median time-to-detection
- median time-to-acknowledgment
- median time-to-corrective-action
- override rate
- harmful false positive rate
- no-action-after-alert rate
- harm per 1,000 encounters in pilot domains
- clinician trust score

15. Database Blueprint

Core Tables

patients

- patient_id
- mrn_hash
- demographics_json
- current_location
- encounter_status

encounters

- encounter_id
- patient_id
- encounter_type
- admission_time
- discharge_time
- unit
- attending_team

actors

- actor_id
- role
- specialty
- unit
- active_status

events

- event_id
- encounter_id
- patient_id
- actor_id
- event_time
- event_type
- source_system
- payload_json
- normalized_concepts_json

orders

- order_id
- encounter_id
- patient_id

- order_type
- order_status
- ordered_at
- medication_code
- dose
- route
- frequency
- indications_json

results

- result_id
- encounter_id
- patient_id
- result_type
- collected_at
- reported_at
- result_value_json
- critical_flag
- acknowledged_at
- acknowledged_by

notes

- note_id
- encounter_id
- patient_id
- note_type
- author_id
- created_at
- raw_text
- extracted_entities_json

devices

- device_id
- patient_id
- encounter_id
- device_type
- stream_type
- installed_at
- removed_at

alerts

- alert_id
- encounter_id

- patient_id
- generated_at
- alert_type
- severity
- confidence_score
- evidence_json
- recommended_action_json
- status
- routed_to_json

alert_actions

- action_id
- alert_id
- acted_by
- acted_at
- action_type
- override_reason
- free_text_comment

process_instances

- process_instance_id
- encounter_id
- process_type
- started_at
- expected_next_step
- state_json
- closure_status

incidents

- incident_id
- encounter_id
- patient_id
- discovered_at
- incident_type
- severity
- preventability_score
- final_review_json

models

- model_id
- model_name
- model_version
- intended_use

- approval_status
- deployed_at
- retired_at

model_runs

- run_id
- model_id
- encounter_id
- input_snapshot_ref
- output_json
- latency_ms
- generated_at

policies

- policy_id
- policy_name
- version
- scope
- logic_json
- approved_by
- activated_at

16. Event Taxonomy

- order_created
- order_modified
- order_cancelled
- med_administered
- lab_collected
- lab_reported
- result_acknowledged
- imaging_report_finalized
- vital_sign_observed
- escalation_triggered
- escalation_completed
- discharge_started
- discharge_completed
- referral_created
- referral_closed
- checklist_started
- checklist_completed
- consent_signed
- note_signed
- device_alarm
- barcode_scan

- handoff_recorded

17. Alert Taxonomy

- information
- recommendation
- precaution
- warning
- urgent escalation
- hard-stop proposal

Each alert should contain: - why this alert fired - what evidence supports it - who should act action is recommended - whether the user can override and how

18. Multi-Agent Build Workflow

Agent A: Founder / Product Architect

Produces PRD, roadmap, risk matrix, scope boundaries.

Agent B: Clinical Knowledge Engineer

Converts guidelines and SOPs into computable rules, pathways, and process graphs.

Agent C: Data and Interoperability Engineer

Builds HL7/FHIR/DICOM connectors, schemas, parsers, terminology mappers.

Agent D: Detection / ML Engineer

Builds rules, anomaly detection, temporal models, NLP pipelines, evaluation harnesses.

Agent E: Safety and Red-Team Engineer

Creates adversarial cases, false-positive challenge sets, workflow failure simulations.

Agent F: Application Engineer

Builds UI, notification flows, review queues, dashboards, mobile layer.

Agent G: DevSecOps / MLOps Engineer

Builds CI/CD, registries, environments, monitoring, rollback, secrets, access control.

Agent H: Validation / Documentation Engineer

Creates model cards, validation reports, SOPs, audit package, pilot docs.

19. Suggested Model Responsibilities

GPT-5.3-Codex

- repo scaffolding
- backend generation
- data connectors
- tests
- CI/CD
- infra scripts

GPT-5.4

- workflow automation
- long-horizon agent orchestration
- legacy UI / system operations where approved
- end-to-end verification of integrated workflows

Claude Opus 4.6

- architecture critique
- deep code review
- policy reasoning
- threat modeling
- long-context synthesis and spec refinement

Gemini family

- multimodal extraction
- image/audio/video understanding
- high-throughput perceptual tasks where validated

20. Governance and Safety Requirements

- human approval for high-risk interventions
- complete auditability
- policy-based release gates
- shadow-mode validation before live use
- override capture and review
- fairness and drift monitoring
- site-specific governance board
- privacy and PHI isolation controls
- data retention and deletion policies

- incident response playbooks

21. Validation Plan

Stage 1: Offline Retrospective Validation

- benchmark historical events
- calculate sensitivity, specificity, PPV, NPV
- assess subgroup performance

Stage 2: Silent Prospective Validation

- run in background
- compare alerts with actual events and clinician actions
- quantify alert burden and timeliness

Stage 3: Controlled Live Pilot

- one unit / one hospital
- limited alert classes
- mandatory review of severe alerts
- weekly governance review

Stage 4: Scale-Up

- multi-unit
- multi-site
- specialty adaptation
- federated or centrally governed learning

22. 12-Month Milestones

Months 0-2

- concept finalization
- stakeholder mapping
- governance charter
- pilot scope lock
- ontology v1

Months 2-4

- data inventory
- connector build
- event schema
- rules v1
- synthetic test suite

Months 4 - 6

- pilot engines v 1
- UI v 1
- alert router
- audit logging
- internal validation

Months 6 - 8

- silent mode deployment
- calibration and suppression tuning
- clinician feedback loop

Months 8 - 1 0

- controlled live pilot
- KPI tracking
- model revision
- safety review board cadence

Months 1 0 - 1 2

- pilot impact analysis
- funding package
- manuscript / abstract prep
- phase- 2 expansion plan

2 3 . Commercialization Paths

- SaaS for private hospital networks
- enterprise license for hospital groups
- public-health safety platform for state systems
- white-label patient-safety layer for EMR vendors
- modular APIs for pharmacy, lab, and diagnostic safety

2 4 . Risks

- poor data quality
- low workflow fit
- alert fatigue
- weak ground truth labels
- liability concerns
- clinician distrust
- multi-site generalization failure
- governance immaturity
- cybersecurity exposure

25. Immediate Founder Actions

1. Finalize pilot scope
2. Build safety ontology v1
3. Prepare stakeholder deck
4. Map required data sources at one pilot site
5. Draft validation protocol
6. Draft implementation budget
7. Build code repository structure
8. Stand up synthetic sandbox
9. Build medication safety rules v1
10. Prepare ethics / governance package

26. Master Prompt Pack

Prompt 1: Master Founder Prompt

You are the lead product architect, clinical informatics strategist, healthcare safety engineer, and fu AI systems planner for Project SentinelCare. Build a founder-grade blueprint for a multimodal medi detection, prediction, prevention, and learning platform. Output: PRD, architecture, domain model, eve model, user stories, pilot use-cases, risk register, governance model, validation plan, and phased ro Assume deployment across hospitals, labs, pharmacies, ICU, OT, ambulatory care, and public-health pathways. Maintain human-in-the-loop safety. Do not assume autonomous medical decision authority.

Prompt 2: Repository Scaffolding Prompt for Codex

Create a production-grade monorepo for Project SentinelCare with services for interoperability, even ingestion, clinical rules, real-time scoring, alert routing, human review queue, governance console, analytics dashboard. Use Python/FastAPI for backend services, PostgreSQL for core storage, Kafka Redpanda for event streaming, React/Next.js for frontend, and docker-compose plus Kubernetes manife Include tests, CI/CD, secrets templates, environment configs, API contracts, and developer docs.

Prompt 3: Clinical Rule Authoring Prompt

Convert the following hospital policy, guideline, or SOP into machine-readable safety rules. For ea output: trigger condition, inclusion criteria, exclusion criteria, evidence sources, severity, recommended action, escalation chain, override policy, and audit fields. Avoid ambiguity. Mark areas requiring cl governance approval.

Prompt 4: Temporal Risk Modeling Prompt

Design models for three pilot use-cases: medication safety, critical-result closure failure, and deterior surveillance. For each, propose labels, feature sets, temporal windows, baseline heuristics, ML meth calibration strategy, validation metrics, and silent-mode deployment plan.

Prompt 5: Safety Red-Team Prompt

Generate 100 adversarial and edge-case scenarios that could cause false positives, false negative unsafe automation bias, workflow disruption, or alert fatigue in Project SentinelCare. Categorize by severity, root cause, and mitigation.

Prompt 6: UI / Workflow Prompt

Design role-based interfaces for nurses, residents, consultants, pharmacists, quality officers, and administrators. Prioritize fast actionability, low cognitive load, traceable evidence, and one-click correct workflows.

Prompt 7: Governance Prompt

Draft a healthcare AI governance package for Project SentinelCare, including approval flows, model audit requirements, validation thresholds, change management, incident response, downtime fallback, ar human oversight requirements.

Prompt 8: Validation Protocol Prompt

Write a retrospective + prospective validation protocol for Project SentinelCare pilots. Include object endpoints, data sources, inclusion criteria, labeling methods, statistical analysis plan, calibration chec subgroup analysis, override analysis, and implementation safety monitoring.

27. Recommended Initial Repo Structure

- /docs
- /services/interoperability
- /services/event-store
- /services/rules-engine
- /services/scoring-engine
- /services/alert-router
- /services/review-queue
- /services/governance-console
- /services/analytics
- /frontend/web
- /frontend/mobile
- /infra/docker
- /infra/k8s
- /ml/notebooks
- /ml/pipelines
- /tests
- /synthetic-data
- /policies

28. Closing Position

SentinelCare should be built as a staged patient-safety platform that begins with narrow, high-valu measurability pilots and expands through validated multimodal modules. The winning strategy is brea architecture, but strict prioritization in deployment.

29. System Architecture Diagram (Text Blueprint)

```
                        +-------------------------------------+
                        |         External Data Sources        |
                        |-------------------------------------|
                        | EHR/EMR | Lab | Radiology | Pharmacy |
                        | ICU/Monitors | OT | Billing | HRMS   |
                        | IoT/Devices | Mobile | Wearables     |
                        +------------------+-------------------+
                                           |
                                           v
+-------------------------------------------------------------------------------
+
|                           INTEROPERABILITY &
INGESTION                           |
|-------------------------------------------------------------------------------|
| HL7/FHIR/DICOM adapters | API gateways | File parsers | Stream
listeners         |
| Terminology mapper      | Identity resolution | Time
normalization               |
+---------------------------------------------
+-----------------------------------+
                                                |
                                                v
+-------------------------------------------------------------------------------
+
|                               UNIVERSAL EVENT
FABRIC                              |
|-------------------------------------------------------------------------------|
| Kafka/Redpanda topics | Event normalization | Event store | Feature
extraction    |
| Audit stream | Process-instance builder | State transition
tracker                |
+---------------------------------------------
+-----------------------------------+
                                                |
                      +-------------------------+-------------------------+
                      |                                                   |
                      v                                                   v
+---------------------------------------------+
```

```
+---------------------------------------------+
|          CLINICAL KNOWLEDGE LAYER           |      |              DATA
SERVICES                   |
|---------------------------------------------|
|---------------------------------------------|
| Policies | SOPs | Formularies | Checklists  |      | PostgreSQL | Object store
| Vector index    |
| Contraindications | Escalation rules        |      | Feature store | Metadata
registry            |
+-------------------+-------------------------+      +-------------------
+-------------------------+
                    |                                                   |
                    +-------------------------+-------------------------+
                                              |
                                              v
+-------------------------------------------------------------------------------
+
|                           DETECTION & PREDICTION
LAYER                           |
|-------------------------------------------------------------------------------|
| Rules engine | Anomaly engine | NLP engine | Vision/audio
extractors             |
| Temporal risk models | Process deviation engine | Contradiction
engine            |
+---------------------------------------------
+-------------------------------------+
                                              |
                                              v
+-------------------------------------------------------------------------------
+
|                             ALERT & INTERVENTION
LAYER                             |
|-------------------------------------------------------------------------------|
| Silent watchlists | Soft alerts | Interruptive alerts |
Escalations              |
| Checklist launchers | One-click corrective actions | Routing
policies            |
+---------------------------------------------
+-------------------------------------+
                                              |
                                              v
+-------------------------------------------------------------------------------
+
|                           EXPERIENCE & OPERATIONS
LAYER                           |
|-------------------------------------------------------------------------------|
| Nurse UI | Resident UI | Consultant UI | Pharmacist UI | Safety Command
Center    |
```

```
| Admin dashboard | Human review queue | Feedback capture | Override
logging         |
+-------------------------------------------
+-----------------------------------+
                                          |
                                          v
+-------------------------------------------------------------------------------
+
|                            GOVERNANCE, MLOPS,
SECURITY                          |
|-------------------------------------------------------------------------------|
| Model registry | Approval workflows | CI/CD | Drift monitoring | Audit
trails      |
| Incident response | Access control | Encryption | Rollback | Kill
switch           |
+-------------------------------------------------------------------------------
+
```

30. Monorepo Folder Tree With Exact Files

```
sentinelcare/
├── README.md
├── LICENSE
├── .gitignore
├── .env.example
├── docker-compose.yml
├── Makefile
├── docs/
│   ├── PRD.md
│   ├── ARCHITECTURE.md
│   ├── GOVERNANCE.md
│   ├── VALIDATION_PROTOCOL.md
│   ├── ALERT_TAXONOMY.md
│   ├── EVENT_MODEL.md
│   ├── DATA_DICTIONARY.md
│   └── RUNBOOKS/
│       ├── incident_response.md
│       ├── rollback.md
│       └── downtime_fallback.md
├── infra/
│   ├── docker/
│   │   ├── api.Dockerfile
│   │   ├── worker.Dockerfile
│   │   ├── frontend.Dockerfile
│   │   └── ml.Dockerfile
```

```
│   ├── k8s/
│   │   ├── namespace.yaml
│   │   ├── postgres.yaml
│   │   ├── kafka.yaml
│   │   ├── api-gateway.yaml
│   │   ├── rules-engine.yaml
│   │   ├── scoring-engine.yaml
│   │   ├── alert-router.yaml
│   │   ├── review-queue.yaml
│   │   ├── frontend.yaml
│   │   └── ingress.yaml
│   └── terraform/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
├── services/
│   ├── api-gateway/
│   │   ├── app/main.py
│   │   ├── app/routes_health.py
│   │   ├── app/routes_alerts.py
│   │   ├── app/routes_patients.py
│   │   ├── app/deps.py
│   │   ├── tests/test_health.py
│   │   └── requirements.txt
│   ├── interoperability/
│   │   ├── app/main.py
│   │   ├── app/hl7_parser.py
│   │   ├── app/fhir_client.py
│   │   ├── app/dicom_listener.py
│   │   ├── app/terminology_mapper.py
│   │   ├── app/identity_resolver.py
│   │   └── requirements.txt
│   ├── event-store/
│   │   ├── app/main.py
│   │   ├── app/models.py
│   │   ├── app/schemas.py
│   │   ├── app/repository.py
│   │   ├── app/consumer.py
│   │   ├── app/producer.py
│   │   └── requirements.txt
│   ├── rules-engine/
│   │   ├── app/main.py
│   │   ├── app/rule_runtime.py
│   │   ├── app/rule_registry.py
│   │   ├── app/medication_rules.py
│   │   ├── app/critical_result_rules.py
```

```
│   │   ├── app/deterioration_rules.py
│   │   ├── app/explanations.py
│   │   └── requirements.txt
│   ├── scoring-engine/
│   │   ├── app/main.py
│   │   ├── app/features.py
│   │   ├── app/model_loader.py
│   │   ├── app/score_pipeline.py
│   │   ├── app/calibration.py
│   │   └── requirements.txt
│   ├── alert-router/
│   │   ├── app/main.py
│   │   ├── app/router.py
│   │   ├── app/escalation.py
│   │   ├── app/notifiers.py
│   │   ├── app/policies.py
│   │   └── requirements.txt
│   ├── review-queue/
│   │   ├── app/main.py
│   │   ├── app/review_api.py
│   │   ├── app/override_api.py
│   │   ├── app/audit_api.py
│   │   └── requirements.txt
│   ├── governance-console/
│   │   ├── app/main.py
│   │   ├── app/model_registry.py
│   │   ├── app/policy_registry.py
│   │   ├── app/approvals.py
│   │   └── requirements.txt
│   ├── analytics/
│   │   ├── app/main.py
│   │   ├── app/kpi_service.py
│   │   ├── app/drift_monitor.py
│   │   ├── app/reporting.py
│   │   └── requirements.txt
│   └── synthetic-sandbox/
│       ├── app/generate_patients.py
│       ├── app/generate_events.py
│       ├── app/generate_alert_truth.py
│       └── requirements.txt
├── frontend/
│   ├── web/
│   │   ├── package.json
│   │   ├── next.config.js
│   │   ├── app/page.tsx
│   │   ├── app/alerts/page.tsx
```

```
│   │   ├── app/watchlist/page.tsx
│   │   ├── app/reviews/page.tsx
│   │   ├── app/governance/page.tsx
│   │   ├── components/AlertCard.tsx
│   │   ├── components/WatchlistTable.tsx
│   │   └── lib/api.ts
│   └── mobile/
│       ├── pubspec.yaml
│       ├── lib/main.dart
│       ├── lib/screens/alerts_screen.dart
│       ├── lib/screens/patient_detail.dart
│       └── lib/services/api_service.dart
├── ml/
│   ├── notebooks/
│   │   ├── 01_medication_safety_baseline.ipynb
│   │   ├── 02_critical_results_baseline.ipynb
│   │   └── 03_deterioration_baseline.ipynb
│   ├── pipelines/
│   │   ├── train_medication.py
│   │   ├── train_critical_results.py
│   │   ├── train_deterioration.py
│   │   └── evaluate_common.py
│   └── model_cards/
│       ├── medication_v1.md
│       ├── critical_results_v1.md
│       └── deterioration_v1.md
├── policies/
│   ├── medication_rules_v1.yaml
│   ├── critical_results_v1.yaml
│   ├── deterioration_v1.yaml
│   └── escalation_policies_v1.yaml
├── tests/
│   ├── integration/
│   │   ├── test_alert_pipeline.py
│   │   ├── test_rules_runtime.py
│   │   └── test_event_ingestion.py
│   └── e2e/
│       ├── test_medication_safety_flow.py
│       ├── test_critical_results_flow.py
│       └── test_deterioration_flow.py
└── scripts/
    ├── bootstrap.sh
    ├── seed_demo_data.py
    └── run_local_stack.sh
```

31. Database ERD (Logical)

```
PATIENTS (1) ----< ENCOUNTERS (many)
PATIENTS (1) ----< EVENTS (many)
PATIENTS (1) ----< ORDERS (many)
PATIENTS (1) ----< RESULTS (many)
PATIENTS (1) ----< NOTES (many)
PATIENTS (1) ----< DEVICES (many)
PATIENTS (1) ----< ALERTS (many)

ENCOUNTERS (1) ----< EVENTS (many)
ENCOUNTERS (1) ----< ORDERS (many)
ENCOUNTERS (1) ----< RESULTS (many)
ENCOUNTERS (1) ----< NOTES (many)
ENCOUNTERS (1) ----< PROCESS_INSTANCES (many)
ENCOUNTERS (1) ----< INCIDENTS (many)

ACTORS (1) ----< EVENTS (many)
ACTORS (1) ----< NOTES (many)
ACTORS (1) ----< ALERT_ACTIONS (many)

ALERTS (1) ----< ALERT_ACTIONS (many)
MODELS (1) ----< MODEL_RUNS (many)
POLICIES (1) ----< ALERTS (many, logical linkage via policy version)
PROCESS_INSTANCES (1) ----< EVENTS (many, logical linkage by
process_instance_id)
```

SQL-leaning Relationship Notes

- `encounters.patient_id -> patients.patient_id`
- `events.encounter_id -> encounters.encounter_id`
- `events.actor_id -> actors.actor_id`
- `orders.encounter_id -> encounters.encounter_id`
- `results.encounter_id -> encounters.encounter_id`
- `notes.author_id -> actors.actor_id`
- `alerts.encounter_id -> encounters.encounter_id`
- `alert_actions.alert_id -> alerts.alert_id`
- `model_runs.model_id -> models.model_id`

32. First-Pass FastAPI + React Scaffold

Backend API Surface (v1)

Core Endpoints

- `GET /health`

- `POST /events/ingest`
- `GET /patients/{patient_id}/timeline`
- `GET /encounters/{encounter_id}/alerts`
- `POST /alerts/{alert_id}/acknowledge`
- `POST /alerts/{alert_id}/override`
- `GET /watchlist/high-risk`
- `POST /rules/reload`
- `GET /governance/models`
- `GET /analytics/kpis`

Example FastAPI Skeleton

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Any, Dict, List, Optional

app = FastAPI(title="SentinelCare API", version="0.1.0")

class EventIn(BaseModel):
    encounter_id: str
    patient_id: str
    actor_id: Optional[str] = None
    event_type: str
    event_time: str
    source_system: str
    payload: Dict[str, Any]

class AlertActionIn(BaseModel):
    acted_by: str
    reason: Optional[str] = None

@app.get("/health")
def health() -> Dict[str, str]:
    return {"status": "ok"}

@app.post("/events/ingest")
def ingest_event(event: EventIn) -> Dict[str, Any]:
    return {"accepted": True, "event_type": event.event_type}

@app.get("/encounters/{encounter_id}/alerts")
def get_alerts(encounter_id: str) -> List[Dict[str, Any]]:
    return []

@app.post("/alerts/{alert_id}/acknowledge")
def acknowledge_alert(alert_id: str, action: AlertActionIn) -> Dict[str, Any]:
    return {"alert_id": alert_id, "status": "acknowledged", "acted_by":
action.acted_by}
```

```
@app.post("/alerts/{alert_id}/override")
def override_alert(alert_id: str, action: AlertActionIn) -> Dict[str, Any]:
    return {"alert_id": alert_id, "status": "overridden", "acted_by":
action.acted_by}
```

Example React/Next.js Page Skeleton

```
'use client';

import { useEffect, useState } from 'react';

type Alert = {
  alert_id: string;
  severity: string;
  alert_type: string;
  status: string;
};

export default function AlertsPage() {
  const [alerts, setAlerts] = useState<Alert[]>([]);

  useEffect(() => {
    fetch('/api/mock-alerts')
      .then((r) => r.json())
      .then((data) => setAlerts(data));
  }, []);

  return (
    <main className="p-6">
      <h1 className="text-2xl font-semibold mb-4">SentinelCare Alerts</h1>
      <div className="space-y-3">
        {alerts.map((a) => (
          <div key={a.alert_id} className="rounded-2xl border p-4 shadow-sm">
            <div className="font-medium">{a.alert_type}</div>
            <div>Severity: {a.severity}</div>
            <div>Status: {a.status}</div>
          </div>
        ))}
      </div>
    </main>
  );
}
```

33. Medication-Safety Pilot Rules v1

Rule 1: Allergy Conflict

- Trigger: new medication order placed
- Check: ordered medication or class matches documented allergy list
- Severity: urgent
- Action: interruptive alert to ordering clinician and pharmacist
- Override: allowed only with documented justification

Rule 2: Severe Drug-Drug Interaction

- Trigger: medication order placed or medication reconciliation completed
- Check: active medication list contains contraindicated interaction pair
- Severity: urgent
- Action: recommend alternative or monitoring plan

Rule 3: Renal Dose Adjustment Omission

- Trigger: medication order placed
- Check: latest eGFR below threshold and ordered dose exceeds allowed renal-adjusted range
- Severity: warning to urgent based on drug
- Action: suggest dose/frequency change

Rule 4: Hepatic Dose Adjustment Omission

- Trigger: medication order placed
- Check: relevant hepatic impairment markers present and high-risk medication ordered
- Severity: warning
- Action: suggest safer dose or alternative

Rule 5: Duplicate Therapeutic Class

- Trigger: second medication ordered within same therapeutic class
- Check: active medication list already contains same class without explicit bridging rationale
- Severity: warning
- Action: verify intent, de-duplicate if unintended

Rule 6: Omitted Critical Home Medication at Admission

- Trigger: admission medication reconciliation completed
- Check: documented long-term critical medication absent from inpatient plan without rationale
- Severity: warning
- Action: prompt clinician review

Rule 7: High-Risk Drug Without Required Lab Baseline

- Trigger: order placed for predefined high-risk drug

- Check: mandatory recent lab missing
- Severity: warning
- Action: prompt lab ordering before or immediately after administration per policy

Rule 8: LASA Name Confusion Risk

- Trigger: medication search or selection event
- Check: selected medication belongs to look-alike/sound-alike list and similar medication recently viewed/selected
- Severity: precaution
- Action: confirmation dialog with indication and dose verification

Rule 9: Route Mismatch

- Trigger: medication order placed
- Check: route incompatible with formulation or patient context
- Severity: urgent for critical mismatches
- Action: block or confirm based on policy

Rule 10: Discharge Medication Reconciliation Mismatch

- Trigger: discharge summary initiated
- Check: unresolved discrepancy between inpatient meds, home meds, and discharge meds
- Severity: warning
- Action: require reconciliation completion before discharge finalization

34. Full Codex Master Prompt Set for Autonomous Repo Generation

Prompt A: Monorepo Generator

Build a production-grade monorepo called SentinelCare for a real-time medical error detection and platform. Include Python FastAPI microservices, PostgreSQL schema, Kafka-based event flow, Next.js frontend, Flutter mobile shell, Docker, Kubernetes manifests, unit tests, integration tests, sample syn data generators, governance docs, and CI/CD. Structure services as api-gateway, interoperability, even store, rules-engine, scoring-engine, alert-router, review-queue, governance-console, and analytics. Add README files and run instructions.

Prompt B: Event Schema and Ingestion

Design canonical event schemas for patient-safety monitoring. Generate pydantic models, SQLAlchemy models, Alembic migrations, and ingestion endpoints for orders, lab results, radiology reports, medic administration, vitals, notes, device alarms, discharge events, and referrals. Normalize timestamps, act and source system metadata.

Prompt C: Medication Safety Rules Engine

Implement a rules engine for medication safety with YAML-configurable rules and explanation out
Include allergy conflict, severe interaction, renal dose adjustment omission, duplicate therapy, high-ri
drug without baseline lab, route mismatch, LASA risk, and discharge reconciliation mismatch. Add
fixtures and sample inputs.

Prompt D: Critical Result Closure Engine

Implement detection logic for critical lab or radiology results that are unacknowledged, acted on
remain pending after discharge. Generate data models, service logic, escalation policies, reminder int
and test scenarios.

Prompt E: Deterioration Surveillance Engine

Create a deterioration surveillance module that consumes vitals, labs, oxygen requirements, nursing
and escalation events. Compute baseline heuristic scores and configurable thresholds for watchlist,
warning, and urgent escalation. Include explainability and suppression logic.

Prompt F: Frontend UI Generator

Build a Next.js clinician interface with pages for live alerts, patient watchlist, encounter timeline,
queue, alert details, governance, and analytics. Use clean production-ready components, loading stat
empty states, evidence panels, and action buttons for acknowledge, escalate, and override.

Prompt G: MLOps and Governance

Generate MLflow-compatible training and registry scripts, model card templates, approval workflow stu
drift monitoring skeletons, alert performance dashboards, and rollback procedures. Add placeholders
security and compliance controls.

Prompt H: Synthetic Sandbox

Create a synthetic data generator that simulates encounters, medication orders, labs, vitals, critical
deterioration trajectories, and alert ground truth. Ensure deterministic seeds, configurable volumes, a
easy local testing.

Prompt I: End-to-End Test Orchestrator

Write end-to-end tests that simulate three pilot flows: medication safety, critical result follow-up,
deterioration escalation. Verify ingestion, alert generation, routing, user action logging, and analytic
output.

Prompt J: Founder Documentation Pack

Generate concise but complete docs: PRD, architecture overview, module responsibilities, API contrac local setup, deployment guide, rule-authoring guide, governance workflow, and pilot validation checklist.